42028: Deep Learning and Convolutional Neural Networks

Assignment 1, Report

Student Name: Deepanshi Bansal
Student ID: 25007269
Subject: 42028, Autumn 2026
Submission Date: 27 March 2026

Generative AI Declaration

I used a generative AI tool to help improve the clarity and structure of my report. It helped me rewrite sentences in a more formal and readable way. However, the implementation, results, and understanding are based on my own work. The tool was useful for saving time and improving grammar, but I made sure to review and understand everything before submitting. Overall, it was helpful as a support tool, but not for generating the actual solution.

1. Introduction

This report presents the implementation and evaluation of two image classification models for recognising handwritten mathematical symbols. The performance of each model is analysed and compared using a provided dataset.

The dataset contains images from 10 different classes, a mix of digits, letters, and mathematical operators. Before training any classifier, three different types of features were extracted from the images to represent them in a way that is more useful for machine learning:

1. **Raw Pixel** Values, the simplest approach, just using the pixel intensities directly
2. **HOG** (Histogram of Oriented Gradients), captures shape and edge direction information
3. **LBP** (Local Binary Patterns), captures local texture patterns

These features were then used to train and test two classifiers:

1. **SVM** (Support Vector Machine), a well-established machine learning classifier
2. **ANN** (Artificial Neural Network), a simple neural network built with Keras

The goal of this report is to document the full process and compare the performance of each feature and classifier combination to understand what works best for this type of classification task.

2. Dataset

The dataset was generated using the tool provided on Canvas based on student ID 25007269. It contains grayscale images of handwritten mathematical symbols across 10 classes. Each class represents a unique symbol, a combination of digits, letters, and operators.

The 10 classes assigned to this student ID are:

4. 4 - digit four
5. 8 - digit eight
6. M - uppercase letter M
7. S - uppercase letter S
8. div - division symbol
9. e - lowercase letter e (as in Euler's number)
10. ldots - ellipsis (...)
11. tan - tangent function
12. times - multiplication symbol
13. u - lowercase letter u

All images were loaded in grayscale and resized to 32x32 pixels so that every image has the same dimensions going into the feature extraction step. The dataset was then split into 80% training and 20% testing using a fixed random seed (random_state = 42) to ensure reproducibility.

Some sample images from each class were visualised during the experiments to understand the data better. One observation from this was that some classes are more visually similar to each other than others- for example, 'e' and 'u' both have similar curved shapes, and 'div' is a small symbol that could potentially be confused with others. This makes the classification task more challenging compared to working with clearly distinct symbols.

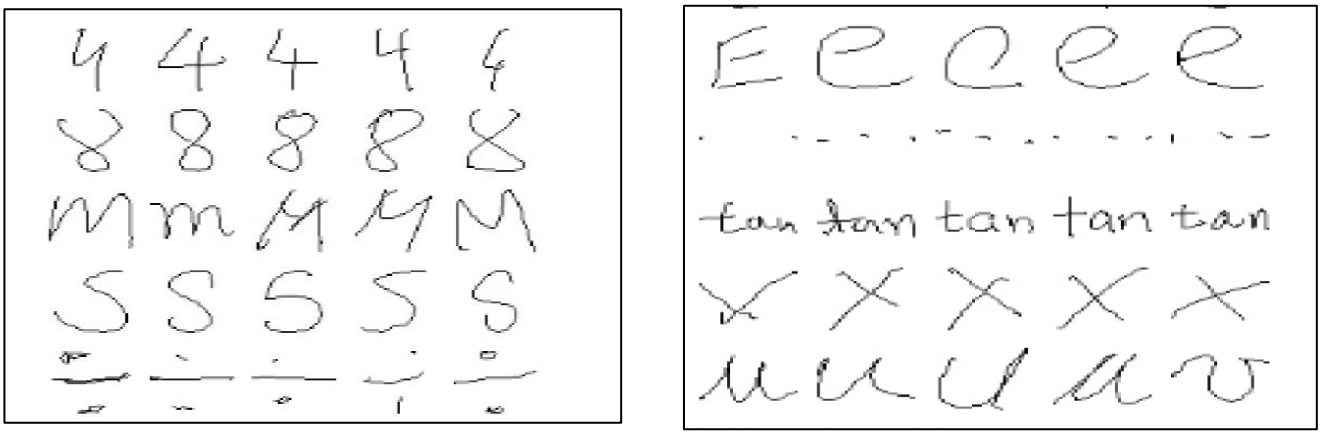

Figure 1: *Sample grayscale images (32×32) from each of the 10 classes in the dataset, showing 5 samples per class.*

3. Experimental Results and Discussion

a. Experimental Settings

The settings used for each classifier are described below.

- **SVM Classifier Settings**

Parameter	Value
Kernel	RBF (Radial Basis Function)
C (Regularisation)	100.0
Random State	42
Feature Scaling	StandardScaler applied before training
Multi-class Strategy	One-vs-One (default in sklearn SVC)

The RBF kernel was chosen because it handles non-linear decision boundaries well, which is important when classifying complex handwritten images. A high C value of 100 was used to minimise training error. StandardScaler was applied to all features before training to normalise the data and prevent any single feature from dominating.

- **ANN Classifier Settings**

Parameter	Value
Input Layer	Size varies by feature (e.g., 1024 for Raw Pixels)
Hidden Layer 1	256 neurons, ReLU activation
Hidden Layer 2	128 neurons, ReLU activation
Output Layer	10 neurons, Softmax activation
Loss Function	Sparse Categorical Cross-Entropy
Optimiser	Adam (default learning rate)
Epochs	30
Batch Size	32
Validation Split	10% of training data
Feature Scaling	StandardScaler applied before training

The ANN was built using TensorFlow and Keras with a Sequential model. ReLU was chosen as the hidden layer activation because it is efficient and works well for most classification tasks. Softmax was used on the output layer because it converts the network outputs into class probabilities, which is standard for multi-class problems. Sparse Categorical Cross-Entropy was used as the loss function because the labels are stored as integers rather than one-hot vectors.

b. Experimental Results

i. Confusion Matrix

The confusion matrix below shows the results for the best-performing combination, which is SVM with HOG features (test accuracy: 0.9860). Each row in the matrix represents the actual class, and each column represents the predicted class. The diagonal values show correct predictions, and any off-diagonal values are misclassifications.

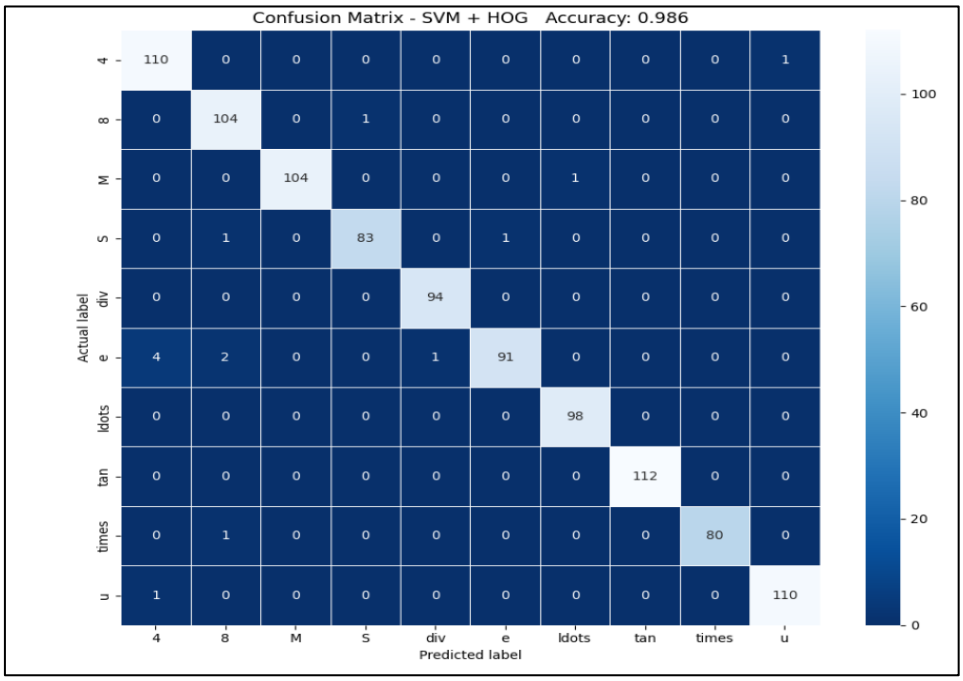

Figure 2: *Confusion matrix for SVM + HOG features (Best performing combination, Accuracy = 0.9860)*

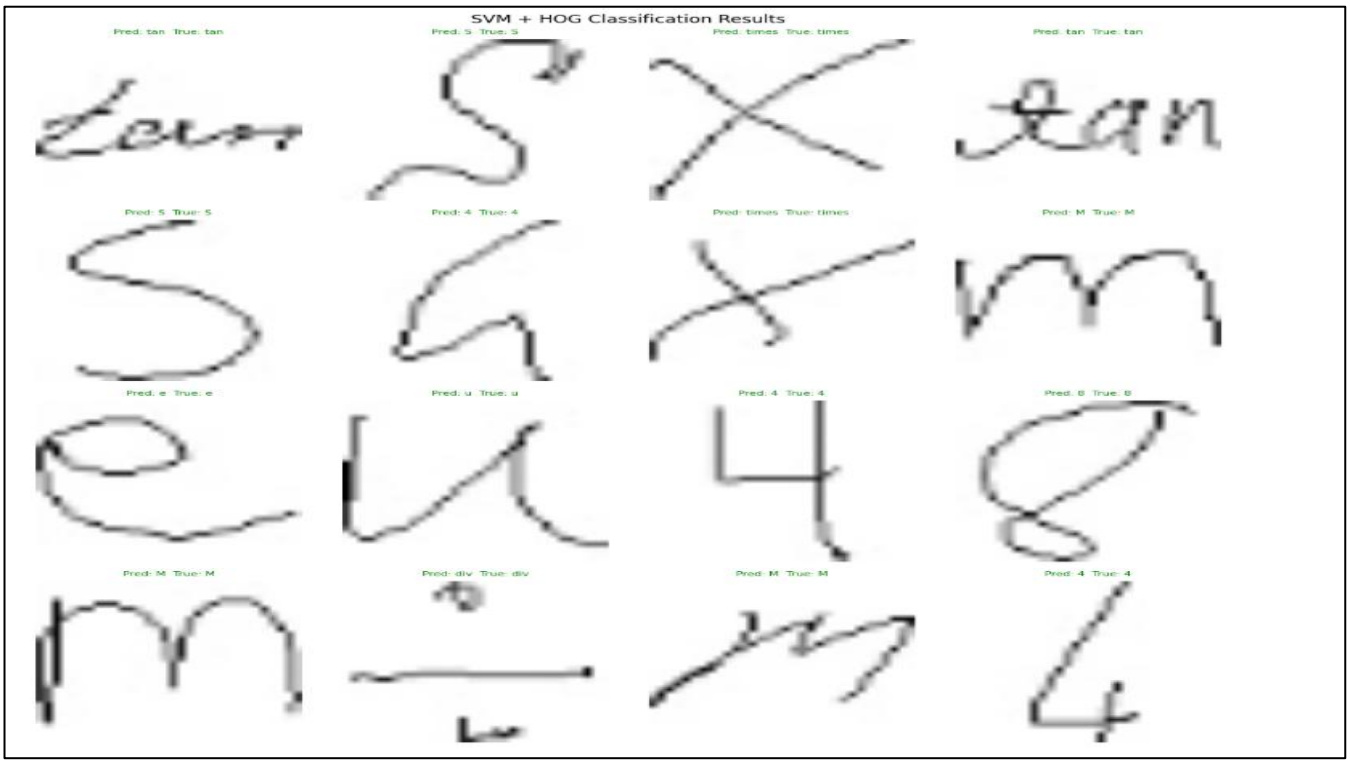

Figure 3: *Sample classification results, green label = correct prediction, red label = incorrect prediction*

From the confusion matrix, it is clear that the classifier is performing well overall. Most samples are correctly classified, as seen by the high values along the diagonal. The few errors that do appear are mostly between classes that look visually similar, such as 'e' and 'u', which is expected given that both have similar rounded shapes when handwritten.

ii. Comparative Study

The table below shows the test accuracy for each combination of classifier and feature type.

Table 1: Comparative test accuracy, all classifier and feature combinations

Classifier / Feature	HOG	LBP	Raw Pixels
SVM	**0.9860**	0.8740	0.9610
ANN	0.9780	0.8500	0.9490

Best performing combination: SVM + HOG (0.9860)

iii. Discussion

Looking at the results, a clear pattern emerges across both classifiers: HOG features consistently give the highest accuracy, followed by Raw Pixels, and then LBP. This pattern makes sense when you think about what each feature actually captures.

HOG works by computing the direction and strength of edges in small local regions of the image. For handwritten symbols, the most important thing that distinguishes one class from another is the shape and direction of the strokes (Dalal & Triggs, 2005). For example, the symbol 'M' has diagonal strokes going in specific directions, while 'tan' looks like three distinct letter shapes grouped together. HOG captures exactly this kind of structural information, which is why it leads to the highest accuracy.

Raw Pixels performed reasonably well but not as well as HOG. The issue with raw pixels is that they are sensitive to small variations in how a symbol is written, a slightly thicker stroke or a small shift in position can result in very different pixel values, even for the same symbol. StandardScaler helps to some extent, but the fundamental limitation remains.

LBP gave the lowest accuracy in both classifiers. LBP captures local texture patterns by comparing neighbouring pixels, which is typically useful for texture-heavy images like faces or material surfaces (Ojala et al., 2002). For handwritten symbols on a plain background, most of the image is white with a few dark strokes. The texture information across different classes is not very distinct, which is why LBP struggles here.

As for the difference between SVM and ANN, SVM with an RBF kernel performed better than ANN across all feature types. This is likely because SVM is very effective at finding optimal decision boundaries in high-dimensional feature spaces, and the dataset size is moderate rather than very large. Handcrafted features such as HOG already provide strong and well-structured representations, which means SVM can make the most of them without requiring a large amount of training data. The ANN, on the other hand, showed signs of overfitting during training: the training accuracy reached close to 100% while the validation accuracy plateaued at a noticeably lower value, particularly with LBP features. This suggests the ANN was memorising the training examples rather than learning generalisable patterns. To address this, techniques such as adding dropout layers, applying early stopping, or reducing the number of training epochs could be used to improve the ANN's generalisation ability.

The misclassifications that do occur are mostly between visually similar classes. For example, 'e' and 'u' both involve curved strokes and can look quite similar when handwritten by different people. Similarly, 'div' is a small symbol that could potentially be confused with other compact symbols. These kinds of errors are expected and are a natural result of working with handwritten data.

4. Conclusion

This assignment involved classifying handwritten mathematical symbols using two classifiers (SVM and ANN) and three feature types (HOG, LBP, and Raw Pixels). Overall, the experiments showed that the choice of feature representation has a significant impact on classification accuracy.

HOG was the best-performing feature across both classifiers because it captures shape and edge direction, which is the most relevant information for distinguishing handwritten symbols. SVM with HOG achieved the highest accuracy of 98.60%, making it the best overall combination in this study. LBP consistently performed the worst because it is a texture-based feature and this dataset is not texture-driven.

SVM slightly outperformed ANN in all cases, which suggests that for this dataset size and with the current ANN architecture, SVM is the more reliable choice. However, the ANN results were still competitive, especially with HOG features at 97.80%. One key limitation observed with the ANN was overfitting: training accuracy reached close to 100%, while validation accuracy was notably lower, indicating the model was not fully generalising. Future work could address this by incorporating dropout layers, applying early stopping, or experimenting with a reduced number of epochs.

This assignment helped build a clear understanding of how traditional feature extraction methods work and why the feature choice matters as much as the classifier itself. A natural next step would be to use a Convolutional Neural Network (CNN), which can learn its own spatial features directly from the images without needing manual feature extraction, and would likely push accuracy even higher.

References

Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. *Proceedings of the 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR),* **1**, 886–893. https://doi.org/10.1109/CVPR.2005.177

Ojala, T., Pietikainen, M., & Maenpaa, T. (2002). Multiresolution gray-scale and rotation invariant texture classification with local binary patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence,* **24**(7), 971–987. https://doi.org/10.1109/TPAMI.2002.1017623

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research,* **12**, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

TensorFlow Developers. (2024). *TensorFlow* (Version 2.18) [Software]. Google. https://www.tensorflow.org

van der Walt, S., Schonberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., & Yu, T. (2014). Scikit-image: Image processing in Python. *PeerJ,* **2**, e453. https://doi.org/10.7717/peerj.453

University of Technology Sydney. (2026). 42028 Deep Learning and Convolutional Neural Networks – Assignment 1 [Course materials]. Canvas. https://canvas.uts.edu.au/courses/38646/modules